

EntryPoint API — Endpoint Testing Guide

Copy-paste curl commands for every endpoint, in the order to run them

Base URL: `http://127.0.0.1:8000/api` | **Seeded login:** `librarian@stidavao.edu.ph / password123`

1. Start MySQL, then reset + seed the database: `php artisan migrate:fresh --seed`
2. Start the API: `php artisan serve`
3. Run the commands below **in order** — later steps reuse IDs/tokens captured from earlier responses.
4. On Windows, run these in **Git Bash** or WSL. In PowerShell, use `curl.exe` explicitly (PowerShell's built-in `curl` alias is not compatible with these flags).

PUBLIC no token needed **ANY** any logged-in user **STUDENT** approved student token **LIBRARIAN** librarian token

1. AUTH & REGISTRATION

[1.1] **POST** `/login` **PUBLIC**

```
curl -X POST "http://127.0.0.1:8000/api/login" \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-d '{"email":"librarian@stidavao.edu.ph","password":"password123"}'
```

Expect: **200** — returns { user, access_token, token_type }.

→ Save access_token as LIB_TOKEN: `export LIB_TOKEN=<paste>`

[1.2] **POST** `/register` **PUBLIC**

```
curl -X POST "http://127.0.0.1:8000/api/register" \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-d '  
{  
  "firstName": "Test",  
  "lastName": "Student",  
  "email": "test_student_1@davao.sti.edu.ph",  
  "password": "password123",  
  "password_confirmation": "password123",  
  "studentIDNumber": "S-0001"  
}'
```

Expect: **201** — registrationStatus pending, barcodeValue is null.

→ Save student.studentID as STUDENT_ID, student.user.email as STUDENT_EMAIL

Optional fields: middleInitial, phoneNumber, birthDate, address, academicProgram

[1.3] **POST** `/librarian/students/{STUDENT_ID}/approve` **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/students/$STUDENT_ID/approve" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** — registrationStatus becomes approved, barcodeValue generated (format STI-XXXXXXXXXX).

→ Save student.barcodeValue as BARCODE

[1.4] **POST** `/librarian/students/{STUDENT_ID}/reject` **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/students/$STUDENT_ID_B/reject" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** — registrationStatus becomes rejected.

Use a DIFFERENT pending student than the one you approved above.

[1.5] **POST** /login **PUBLIC**

```
curl -X POST "http://127.0.0.1:8000/api/login" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-d '{"email": "$STUDENT_EMAIL", "password": "password123"}'
```

Expect: **200** once approved (401/422 with a "pending approval" message before 1.3 runs).

→ Save `access_token` as `STU_TOKEN`

[1.6] **GET** /user **ANY**

```
curl -X GET "http://127.0.0.1:8000/api/user" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — returns the logged-in user's own profile.

[1.7] **POST** /logout **ANY**

```
curl -X POST "http://127.0.0.1:8000/api/logout" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — "Logged out successfully." The token used is now invalid.

2. NOTIFICATIONS (GENERIC, USED BY EVERY ROLE)

[2.1] **GET** /notifications **ANY**

```
curl -X GET "http://127.0.0.1:8000/api/notifications" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — { notifications: [...], unreadCount: N }.

[2.2] **PATCH** /notifications/{NOTIFICATION_ID}/read **ANY**

```
curl -X PATCH "http://127.0.0.1:8000/api/notifications/$NOTIFICATION_ID/read" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — that notification's `isRead` becomes true. 403 if it belongs to another user.

[2.3] **PATCH** /notifications/read-all **ANY**

```
curl -X PATCH "http://127.0.0.1:8000/api/notifications/read-all" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — all of the caller's notifications marked read.

3. BOOK CATALOG (LIBRARIAN MANAGES)

[3.1] **POST** /librarian/books **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/books" \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN" \  
-d '{"title":"Test Driven  
Laravel","classNumber":"QA76.T3","quantity":1,"areasOfLibrary":"circulation","subjectName":"Software Engineering","authors":  
[{"name":"Ada Lovelace"}]}'
```

Expect: **201** — returns book with copies[] (one BookCopy per quantity), each with a generated barcodeValue and accessionNumber.

→ Save book.bookID as **BOOK_ID**, book.copies[0].copyID as **COPY_ID**

areasOfLibrary options: circulation, reserved, filipiniana, fiction, thesis, journal, dissertation. Use "subjectID" or "authorID" instead of names to reuse existing rows.

[3.2] **PATCH** /librarian/books/{BOOK_ID} **LIBRARIAN**

```
curl -X PATCH "http://127.0.0.1:8000/api/librarian/books/$BOOK_ID" \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN" \  
-d '{"title":"New Title","quantity":3}'
```

Expect: **200** — updated book. Raising quantity adds new copies; lowering it retires available copies (422 if not enough are available to retire).

[3.3] **DELETE** /librarian/books/{BOOK_ID} **LIBRARIAN**

```
curl -X DELETE "http://127.0.0.1:8000/api/librarian/books/$BOOK_ID" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** if no copies are currently borrowed, else **422** ("N copy(ies) currently borrowed").

4. WISHLIST (STUDENT) — SAVE-FOR-LATER, SEPARATE FROM THE CART

[4.1] **GET** /student/wishlist **STUDENT**

```
curl -X GET "http://127.0.0.1:8000/api/student/wishlist" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200**.

[4.2] **POST** /student/wishlist **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/wishlist" \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $STU_TOKEN" \  
-d '{"bookID":"$BOOK_ID}"'
```

Expect: **201**. 422 if that book is already on the wishlist.

[4.3] **DELETE** /student/wishlist/{WISHLIST_ID} **STUDENT**

```
curl -X DELETE "http://127.0.0.1:8000/api/student/wishlist/${WISHLIST_ID}" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200**. 403 if it belongs to another student.

5. CART → RESERVATIONS (STUDENT), ACCEPT/REJECT (LIBRARIAN)

[5.1] **POST** /student/cart **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/cart" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $STU_TOKEN" \
-d '{"bookID": "$BOOK_ID"}'
```

Expect: **201**. 422 if cart is full (max 3) or already in cart.

[5.2] **GET** /student/cart **STUDENT**

```
curl -X GET "http://127.0.0.1:8000/api/student/cart" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200**.

[5.3] **DELETE** /student/cart/{WISHLIST_ID} **STUDENT**

```
curl -X DELETE "http://127.0.0.1:8000/api/student/cart/${WISHLIST_ID}" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — removes from cart only, stays on wishlist if it was there.

[5.4] **POST** /student/reservations **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/reservations" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **201** — one Reservation per cart item, status Waiting, clears the cart. 422 if cart is empty, over the 3-active cap, or already actively reserved.

→ Save `reservations[0].reservationID` as `RESERVATION_ID`

[5.5] **GET** /student/reservations **STUDENT**

```
curl -X GET "http://127.0.0.1:8000/api/student/reservations" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — each Waiting reservation includes `queuePosition` (1 = next).

[5.6] **DELETE** /student/reservations/{RESERVATION_ID} **STUDENT**

```
curl -X DELETE "http://127.0.0.1:8000/api/student/reservations/${RESERVATION_ID}" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** only if status is Waiting (422 otherwise). 403 if not yours.

[5.7] **GET** /librarian/reservations **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/reservations?status=Waiting" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**.

Optional query: ?status=Waiting|Accepted|Rejected|Fulfilled

[5.8] **GET** /librarian/reservations/queue/{BOOK_ID} **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/reservations/queue/$BOOK_ID" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** — ordered list of Waiting reservations for that book (FIFO).

[5.9] **POST** /librarian/reservations/{RESERVATION_ID}/accept **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/reservations/$RESERVATION_ID/accept" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** only if this is the earliest Waiting reservation for the book AND a copy is available. 422 otherwise. Notifies the student "ready for pickup".

[5.10] **POST** /librarian/reservations/{RESERVATION_ID}/reject **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/reservations/$RESERVATION_ID_B/reject" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** (only from Waiting). Notifies the student it was declined.

6. LOANS (LIBRARIAN CHECKOUT/RETURN), SELF-RETURN (STUDENT + LIBRARIAN)

[6.1] **POST** /librarian/loans **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/loans" \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN" \  
-d '{"studentID": "$STUDENT_ID", "copyID": "$COPY_ID", "reservationID": "$RESERVATION_ID"}'
```

Expect: **201** — dueDate computed from config/loans.php by areasOfLibrary (circulation=2 days, reserved=overnight, filipiniana=3 days, fiction=5 days; thesis/journal/dissertation not loanable — 422).

→ Save loan.loanID as LOAN_ID

"reservationID" is optional — converts an Accepted reservation into a loan.

[6.2] **POST** /librarian/loans/{LOAN_ID}/return **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/loans/$LOAN_ID/return" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** only if status is Active. Frees the copy, accrues any final penalty, notifies next student in queue.

[6.3] **POST** /student/loans/{LOAN_ID}/self-return **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/loans/$LOAN_ID/self-return" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **201** — creates a Pending SelfReturnReport. 422 if loan isn't Active or a report is already Pending.

→ Save `report.reportID` as `REPORT_ID`

[6.4] **GET** /librarian/self-return-reports **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/self-return-reports" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** — lists Pending reports only.

[6.5] **POST** /librarian/self-return-reports/{REPORT_ID}/verify **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/self-return-reports/$REPORT_ID/verify" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** (only from Pending). Marks Verified and finalizes the return exactly like 6.2.

[6.6] **POST** /librarian/self-return-reports/{REPORT_ID}/reject **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/self-return-reports/$REPORT_ID/reject" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** (only from Pending). Student can resubmit after a 6-hour cooldown.

7. RESOURCES (COMPUTER UNITS, STUDY AREAS) + USAGE SESSIONS

[7.1] **POST** /librarian/resources **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/resources" \  
-H "Accept: application/json" \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN" \  
-d '{"resourceType":"computer","name":"Computer 1"}'
```

Expect: **201** — status defaults to Available.

→ Save `resource.resID` as `RES_ID`

[7.2] **GET** /librarian/resources **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/resources?resourceType=computer&status=Available" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** — each resource includes `activeUsage` (who's using it now, if anyone).

[7.3] **PATCH** /librarian/resources/{RES_ID} **LIBRARIAN**

```
curl -X PATCH "http://127.0.0.1:8000/api/librarian/resources/$RES_ID" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $LIB_TOKEN" \
-d '{"name": "Computer 1B", "status": "Unavailable"}'
```

Expect: **200**. status may only be set to Available/Unavailable manually (never "In Use"), blocked while currently In Use.

[7.4] **DELETE** /librarian/resources/{RES_ID} **LIBRARIAN**

```
curl -X DELETE "http://127.0.0.1:8000/api/librarian/resources/$RES_ID" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**, 422 if currently In Use.

[7.5] **GET** /student/resources **STUDENT**

```
curl -X GET "http://127.0.0.1:8000/api/student/resources" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — read-only availability list (no session control from here).

[7.6] **POST** /librarian/resource-usage-logs **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/resource-usage-logs" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $LIB_TOKEN" \
-d '{"resID": "$RES_ID", "studentID": "$STUDENT_ID"}'
```

Expect: **201** — resource flips to In Use, student notified. 422 if resource isn't Available or student already has an open session.

→ Save `usageLog.usageID` as `USAGE_ID`

[7.7] **GET** /librarian/resource-usage-logs **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/resource-usage-logs?resID=$RES_ID&active=1" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**.

[7.8] **POST** /librarian/resource-usage-logs/{USAGE_ID}/end **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/resource-usage-logs/$USAGE_ID/end" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** — resource flips back to Available. 422 if already ended.

8. ATTENDANCE (BARCODE SCAN, TOGGLES CHECK-IN/CHECK-OUT)

[8.1] **POST** /librarian/attendance-logs/scan **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/attendance-logs/scan" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $LIB_TOKEN" \
-d '{"barcodeValue":"$BARCODE"}'
```

Expect (1st scan of the day): **201** "Checked in." Updates visitStreak on first check-in of a new day (can auto-unlock achievements). Expect (2nd scan): **200** "Checked out." 404 if barcode doesn't match an approved student.

[8.2] **GET** /librarian/attendance-logs **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/attendance-logs?studentID=$STUDENT_ID&active=1" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**.

9. ACHIEVEMENTS (AUTO-UNLOCK VIA CRITERIA, MANUAL REDEEM FOR POINTS)

[9.1] **POST** /librarian/achievements **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/achievements" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $LIB_TOKEN" \
-d '{"name":"First Visit","criteriaJSON":{"metric":"visitStreak","threshold":1},"pointsReward":20}'
```

Expect: **201**. Immediately re-checks every student against this new achievement and auto-unlocks it for anyone who already qualifies.

→ Save achievement.achievementID as **ACHIEVEMENT_ID**

criteriaJSON.metric must be "knowledgeScore" or "visitStreak".

[9.2] **GET** /librarian/achievements **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/achievements" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** — each row includes unlockedCount and redeemedCount.

[9.3] **PATCH** /librarian/achievements/{ACHIEVEMENT_ID} **LIBRARIAN**

```
curl -X PATCH "http://127.0.0.1:8000/api/librarian/achievements/$ACHIEVEMENT_ID" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $LIB_TOKEN" \
-d '{"pointsReward":30}'
```

Expect: **200**. Changing criteriaJSON re-runs the retroactive unlock check.

[9.4] **DELETE** /librarian/achievements/{ACHIEVEMENT_ID} **LIBRARIAN**

```
curl -X DELETE "http://127.0.0.1:8000/api/librarian/achievements/$ACHIEVEMENT_ID" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**.

[9.5] **GET** /student/achievements **STUDENT**

```
curl -X GET "http://127.0.0.1:8000/api/student/achievements" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — every achievement with this student's status: Locked, Unlocked, or Redeemed.

To see "Unlocked", first trigger the underlying metric — e.g. an attendance scan (8.1) bumps visitStreak to 1.

[9.6] **POST** /student/achievements/{ACHIEVEMENT_ID}/redeem **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/achievements/$ACHIEVEMENT_ID/redeem" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** only if unlocked and not yet redeemed. Adds pointsReward to knowledgeScore. 422 otherwise.

10. POINT SHOP — MARKET ITEMS, CART, REDEMPTIONS

[10.1] **POST** /librarian/market-items **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/market-items" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $LIB_TOKEN" \
-d '{"name":"Pencil","type":"supply","pointCost":5,"stock":10}'
```

Expect: **201**.

→ Save item.itemID as ITEM_ID

[10.2] **GET** /librarian/market-items **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/market-items" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**.

[10.3] **PATCH** /librarian/market-items/{ITEM_ID} **LIBRARIAN**

```
curl -X PATCH "http://127.0.0.1:8000/api/librarian/market-items/$ITEM_ID" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $LIB_TOKEN" \
-d '{"stock":0}'
```

Expect: **200**.

Setting stock=0 is a clean way to "retire" an item.

[10.4] **DELETE** /librarian/market-items/{ITEM_ID} **LIBRARIAN**

```
curl -X DELETE "http://127.0.0.1:8000/api/librarian/market-items/$ITEM_ID" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**, or 422 if it has any redemption history (retire via stock=0 instead).

[10.5] **GET** /student/market-items **STUDENT**

```
curl -X GET "http://127.0.0.1:8000/api/student/market-items" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — { points, items: [...] }, each item flagged inStock and affordable.

[10.6] **POST** /student/market-items/{ITEM_ID}/redeem **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/market-items/$ITEM_ID/redeem" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **201** — immediate single-item redemption. 422 if out of stock or not enough points.

[10.7] **POST** /student/market-cart **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/market-cart" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $STU_TOKEN" \
-d '{"itemID": "$ITEM_ID", "quantity": 2}'
```

Expect: **201**.

quantity optional (default 1); adding the same item again just increases its quantity.

[10.8] **GET** /student/market-cart **STUDENT**

```
curl -X GET "http://127.0.0.1:8000/api/student/market-cart" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — { points, cart: [...], total }.

[10.9] **PATCH** /student/market-cart/{CART_ITEM_ID} **STUDENT**

```
curl -X PATCH "http://127.0.0.1:8000/api/student/market-cart/$CART_ITEM_ID" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $STU_TOKEN" \
-d '{"quantity": 5}'
```

Expect: **200**.

[10.10] **DELETE** /student/market-cart/{CART_ITEM_ID} **STUDENT**

```
curl -X DELETE "http://127.0.0.1:8000/api/student/market-cart/$CART_ITEM_ID" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200**.

[10.11] **POST** /student/market-cart/checkout **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/market-cart/checkout" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **201** — redeems every cart line in one transaction (all-or-nothing), clears the cart, one redemption row per line.

→ Save `redemptions[0].redemptionID` as `REDEMPTION_ID`

[10.12] **GET** /librarian/point-redemptions **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/point-redemptions?fulfillmentStatus=Pending" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**.

[10.13] **POST** /librarian/point-redemptions/{REDEMPTION_ID}/fulfill **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/point-redemptions/$REDEMPTION_ID/fulfill" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** (only from Pending).

[10.14] **POST** /librarian/point-redemptions/{REDEMPTION_ID}/cancel **LIBRARIAN**

```
curl -X POST "http://127.0.0.1:8000/api/librarian/point-redemptions/$REDEMPTION_ID/cancel" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** (only from Pending). Refunds points AND restocks the item.

Use a different Pending redemption than the one fulfilled in 10.13.

11. BOOK SUGGESTIONS

[11.1] **POST** /student/book-suggestions **STUDENT**

```
curl -X POST "http://127.0.0.1:8000/api/student/book-suggestions" \
-H "Accept: application/json" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $STU_TOKEN" \
-d '{"title":"Clean Architecture","author":"Robert C. Martin","reason":"Would help with our capstone design"}'
```

Expect: **201** — status Pending, progressStep Submitted.

→ Save `suggestion.suggestionID` as `SUGGESTION_ID`

author and reason are optional.

[11.2] **GET** /student/book-suggestions **STUDENT**

```
curl -X GET "http://127.0.0.1:8000/api/student/book-suggestions" \
-H "Accept: application/json" \
-H "Authorization: Bearer $STU_TOKEN"
```

Expect: **200** — only this student's own suggestions.

[11.3] **GET** /librarian/book-suggestions **LIBRARIAN**

```
curl -X GET "http://127.0.0.1:8000/api/librarian/book-suggestions?status=Pending" \
-H "Accept: application/json" \
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200**.

[11.4]

POST

/librarian/book-suggestions/{SUGGESTION_ID}/approve

LIBRARIAN

```
curl -X POST "http://127.0.0.1:8000/api/librarian/book-suggestions/$SUGGESTION_ID/approve" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** (only from Pending). Does NOT auto-add the book to the catalog — cataloging is a separate manual step (section 3).

[11.5]

POST

/librarian/book-suggestions/{SUGGESTION_ID}/reject

LIBRARIAN

```
curl -X POST "http://127.0.0.1:8000/api/librarian/book-suggestions/$SUGGESTION_ID/reject" \  
-H "Accept: application/json" \  
-H "Authorization: Bearer $LIB_TOKEN"
```

Expect: **200** (only from Pending).

Re-running this guide: most POST actions are one-shot per record (a Waiting reservation can only be accepted once, a Pending report only verified once, etc). To repeat a full pass, reset the database first: `php artisan migrate:fresh --seed` — this restores just the seeded librarian with a clean slate. IDs and tokens are different every time you reset, so always re-export them from each response rather than hardcoding values from a previous run.

Prefer a GUI? Import `hoppscotch/entrypoint-collection.json` and `hoppscotch/entrypoint-local-environment.json` into Hoppscotch, or run the whole suite headlessly with: `npx @hoppscotch/cli test hoppscotch/entrypoint-collection.json -e hoppscotch/entrypoint-local-environment.json`